

PRÁCTICA 8.1

ClassicModels 2.0: Integridad y Blindaje de Datos

CICLO FORMATIVO

TÉCNICO EN DAW

Noel David Núñez Martínez

César Valverde Pardo



CURSO 2025-2026

ÍNDICE

Introducción	3
Triggers	3
Ejercicio 1	3
Ejercicio 2	3
Ejercicio 3	4
Ejercicio 4	4
Ejercicio 5	5
Ejercicio 6	6
Ejercicio 7	6
Ejercicio 8	7
Eventos	7
Ejercicio 9 - Evento 1	7
Ejercicio 10 - Evento 2	8
Conclusiones	9

Introducción

En esta práctica vamos a realizar una serie de ejercicios de triggers y eventos para aprender su realización e implementación con las herramientas y conocimientos adquiridos con el tiempo.

Triggers

Ejercicio 1

Crear un trigger (AFTER INSERT) llamado `updated_stock` que actualice el stock del producto al insertar un pedido. Lo que ha de hacer el trigger es coger el valor `quantityOrdered` y el valor `productCode` de la nueva fila (NEW) insertada en la tabla `orderdetails` y con esos valores actualizar en la tabla `products` el stock correspondiente al producto en cuestión

Sentencia SQL

```
SQL
CREATE DEFINER = CURRENT_USER TRIGGER `classicmodels`.`updated_stock`
AFTER INSERT ON `orderdetails` FOR EACH ROW
BEGIN
    UPDATE products
    SET quantityInStock = quantityInStock - new.quantityOrdered
    WHERE productCode = new.productCode;
END
```

Explicación: Al hacer el trigger, le decimos que de la tabla de `products` se actualice y que el nuevo Stock sea la cantidad en stock menos el `quantityOrdered` nuevo, con el `WHERE` lo condicionamos

Ejercicio 2

Crear un trigger (BEFORE INSERT) llamado `price_checking`. El disparador se asegurará de que el precio estimado de Venta de un producto (MSRP) sea mayor que el precio de costo (`buyPrice`) siempre que se inserten productos en la tabla de productos. De lo contrario, el usuario de la base de datos obtendrá un error.

Sentencia SQL

```
SQL
CREATE DEFINER = CURRENT_USER TRIGGER `price_checking`. BEFORE INSERT ON
`orderdetails` FOR EACH ROW
BEGIN
    IF NEW.MSRP < NEW.buyPrice
```

```

THEN
SIGNAL SQLSTATE '45000'
SET MESSAGE_TEXT = 'MSRP mayor que el buyPrice';
ENDIF;

END

```

Explicación: Antes de insertar los nuevos valores, hacemos una comprobación de que si NEW.MSRP es menor que NEW.buyPrice, que salte el error

Ejercicio 3

A continuación, se creará un (BEFORE UPDATE) trigger denominado product_name_validator. Este disparador impedirá a los usuarios editar el nombre de un producto que ya ha sido insertado en la base de datos. Si existen múltiples usuarios trabajando en la bbdd, un BEFORE UPDATE trigger puede ser utilizado para tener valores de solo lectura y esto puede evitar acciones maliciosas o accidentes de usuarios que puedan modificar registros innecesariamente.

Sentencia SQL

```

SQL
CREATE DEFINER=`root`@`localhost` TRIGGER
`classicmodels`.`product_name_validator` BEFORE UPDATE ON `products` FOR
EACH ROW
BEGIN
    IF NEW.productName <> OLD.productName THEN
        SIGNAL SQLSTATE '45000'
        SET MESSAGE_TEXT = 'Error, el nombre nombre ya fue insertado';
    END IF;
END

```

Explicación: Antes de insertar los nuevos valores, si el nombre nuevo del producto es distinto al antiguo, que salte un error

Ejercicio 4

Creando un (AFTER DELETE) trigger llamado product_archiver. En algunos casos, se requiere guardar información acerca de una acción específica en la bbdd. Indique las instrucciones para que cada vez que se borre un producto se almacene dicha fila en una tabla denominada “ archived_products”.

Sentencia SQL

```

SQL
CREATE DEFINER=`root`@`localhost` TRIGGER
`classicmodels`.`products_AFTER_DELETE` AFTER DELETE ON `products` FOR
EACH ROW
BEGIN
    INSERT INTO archived_products(

```

```

        productCode,
        productName,
        productLine,
        productScale,
        productVendor,
        productDescription,
        quantityInStock,
        buyPrice,
        MSRP) VALUES (
        old.productCode,
        old.productName,
        old.productLine,
        old.productScale,
        old.productVendor,
        old.productDescription,
        old.quantityInStock,
        old.buyPrice,
        old.MSRP);
END

```

Explicación: Le indicamos que antes de borrar, que inserte los valores en una nueva tabla llamada archived_products que se inserten todos los valores antiguos en esta nueva tabla

Ejercicio 5

Control de Stock Mínimo (BEFORE INSERT) Enunciado: Impedir que se procese un detalle de pedido (orderdetails) si la cantidad solicitada es mayor al stock disponible en la tabla products.

Sentencia SQL

```

SQL
CREATE DEFINER=`root`@`localhost` TRIGGER
`classicmodels`.`orderdetails_Stock_Control` BEFORE INSERT ON
`orderdetails` FOR EACH ROW
BEGIN
    DECLARE auxiliar INT;

    SELECT quantityInStock INTO auxiliar
    FROM products
    WHERE productCode = NEW.productCode;

    IF auxiliar < NEW.quantityOrdered THEN
        SIGNAL SQLSTATE '45000'
        SET MESSAGE_TEXT = 'La cantidad pedida es mayor que la que hay en
        Stock';
    END IF;
END

```

```
END IF;  
END
```

Explicación: Buscamos el quantityInStock y lo guardamos en una variable auxiliar, luego le condicionamos, en caso de que auxiliar (quantityInStock) sea menor a la nueva cantidad pedida, que salte el error

Ejercicio 6

Validación de Fecha de Envío (BEFORE UPDATE) Enunciado: En la tabla orders, asegurar que la fecha de envío (shippedDate) no sea anterior a la fecha en la que se realizó el pedido (orderDate).

Sentencia SQL

```
SQL  
CREATE DEFINER=`root`@`localhost` TRIGGER  
`classicmodels`.`orders_BEFORE_UPDATE` BEFORE UPDATE ON `orders` FOR EACH  
ROW  
BEGIN  
    IF NEW.shippedDate IS NOT NULL AND NEW.shippedDate < NEW.orderdate  
    THEN  
        SIGNAL SQLSTATE '45000'  
        SET MESSAGE_TEXT = 'La fecha de envío no puede ser anterior a la  
fecha del pedido';  
    END IF;  
END
```

Explicación: Aquí le decimos que si la nueva fecha de entrega, es menos reciente que la fecha de pedido, que salte el error

Ejercicio 7

Límite de Crédito Excedido (BEFORE INSERT) Enunciado: Al insertar un pago en la tabla payments, verificar que el cliente no esté intentando realizar un pago por un monto superior a su propio límite de crédito (creditLimit) multiplicado por 2 (como política de seguridad ante pagos erróneos).

Sentencia SQL

```
SQL  
CREATE DEFINER=`root`@`localhost` TRIGGER  
`classicmodels`.`payments_BEFORE_INSERT` BEFORE INSERT ON `payments` FOR  
EACH ROW  
BEGIN  
    DECLARE cantidadLimite DECIMAL(10,2);  
    SELECT creditLimit INTO cantidadLimite  
    FROM customers
```

```

WHERE NEW.customerNumber = customerNumber;

IF (cantidadLimite*2) < NEW.amount THEN
SIGNAL SQLSTATE '45000'
SET MESSAGE_TEXT = 'Se supera el doble del tope del limite de
crédito';
END IF;
END

```

Explicación: Declaramos la cantidadLimite como variable auxiliar para ahí guardar la cantidad del creditLimit, donde el customerNumber nuevo debe ser el customerNumber de la tabla, y una vez hecho eso, condicionamos a que si cantidad Limite x2 es menor a la cantidad nueva, que salte el error

Ejercicio 8

Evitar que un empleado sea su propio jefe. El trigger debe lanzar un error si en la tabla employees el campo reportsTo intenta actualizarse con el mismo valor que el employeeNumber

Sentencia SQL

```

SQL
CREATE DEFINER=`root`@`localhost` TRIGGER
`classicmodels`.`employees_BEFORE_UPDATE` BEFORE UPDATE ON `employees` FOR
EACH ROW
BEGIN
IF NEW.employeeNumber = NEW.reportsTo THEN
SIGNAL SQLSTATE '45000'
SET MESSAGE_TEXT = 'No puedes ser tu propio Jefe';
END IF;
END

```

Explicación: Le decimos que si el employeeNumber nuevo es igual al reportsTo nuevo, que le digamos que no puedes ser tu propio jefe

Eventos

Ejercicio 9 - Evento 1

Enunciado: Automatización del Mantenimiento de Sesiones Contexto: En nuestra aplicación web, la tabla sesiones_activas almacena los tokens de acceso de los usuarios. Para optimizar el rendimiento de la base de datos y garantizar la seguridad, es necesario eliminar las sesiones que han permanecido inactivas durante un periodo prolongado.

Sentencia SQL

SQL

```
-- Activar el planificador de eventos
SET GLOBAL event_scheduler = ON;

-- Crear el evento solicitado
CREATE EVENT IF NOT EXISTS e_limpieza_sesiones_inactivas
ON SCHEDULE EVERY 1 HOUR
COMMENT 'Limpia sesiones inactivas para optimizar la tabla
sesiones_activas'
DO
BEGIN
    -- Lógica sencilla: eliminar las sesiones que han permanecido
    inactivas durante un periodo prolongado
    DELETE FROM sesiones_activas
    WHERE ultima_actividad < NOW() - INTERVAL 1 HOUR;
END;
```

Ejercicio 10 - Evento 2

Gestión Automática de Ofertas Expiradas Contexto: En nuestra plataforma de e-commerce, disponemos de una tabla llamada cupones_descuento. Cada cupón tiene una columna fecha_expiracion. Para evitar que los usuarios utilicen códigos obsoletos y mantener la base de datos limpia, debemos desactivar automáticamente los cupones cuya fecha de validez haya pasado

Sentencia SQL

SQL

```
-- 1. Activar el planificador de eventos
SET GLOBAL event_scheduler = ON;

-- 2. Crear el evento de actualización de cupones
CREATE EVENT IF NOT EXISTS e_desactivar_cupones_caducados
ON SCHEDULE EVERY 1 DAY
STARTS (CURRENT_DATE + INTERVAL 1 DAY + INTERVAL 3 HOUR)
COMMENT 'Desactiva cupones que han superado su fecha de validez'
DO
BEGIN
    -- Desactivar automáticamente los cupones cuya fecha de validez haya
    pasado
    UPDATE cupones_descuento
    SET estado = 'Expirado'
    WHERE fecha_expiracion < CURRENT_DATE
    AND estado = 'Activo';
END;
```


Conclusiones

En esta práctica se han utilizado triggers y *eventos* en MySQL para automatizar tareas y mantener la integridad de los datos. Los triggers permiten controlar operaciones en la base de datos y los eventos ejecutar acciones periódicas automáticamente. En conjunto, mejoran la seguridad y el mantenimiento de los datos.